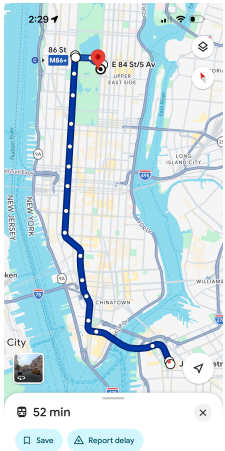
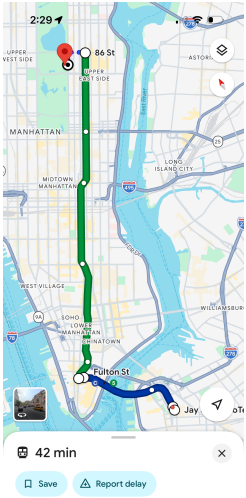
CS-GY 6033
Design and Analysis of Algorithms I

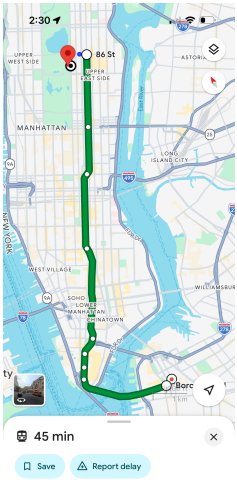
Lecture 8 | Part 1

Shortest Paths in Weighted Graphs

Google Maps


2:29
86 St
M86+
E 84 St/5 Av
52 min
Save
Report delay


2:29
86 St
MANHATTAN
Fulton St
42 min
Save
Report delay


2:30
86 St
MANHATTAN
45 min
Save
Report delay


2:29
86 St
MANHATTAN
14 St - Union Sq
Broadway-Lafayette St
New York
42 min
Save
Report delay

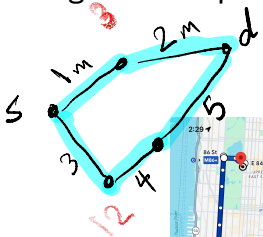
Weighted Graphs

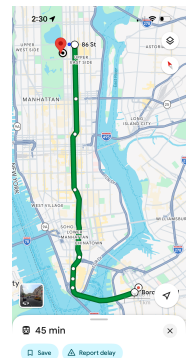
An **edge weighted graph** $G = (V, E, \omega)$ is a triple where (V, E) is a graph and $\omega : E \to \mathbb{R}$ maps each edge to a **weight**.

- Can be directed or undirected.
- In general, weights can be positive, negative, zero.
- Many uses, such as representing **metric spaces**.

Path Lengths

The **length** of a path in a **weighted graph** (usually) refers to the total weight of all edges in the path.



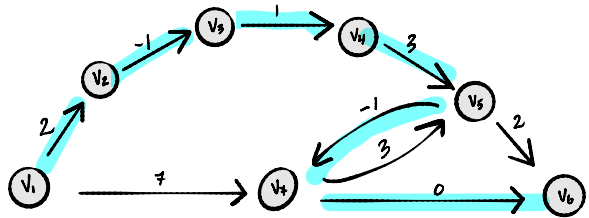


Shortest Paths

- A **shortest path** between u and v is a path between u and v with minimum length.
  - In other words, minimum total weight.

Example

What is the shortest path from v_1 to v_6?



Path: v_1 v_2 v_3 v_4 v_5 v_7 v_6

Length: 4

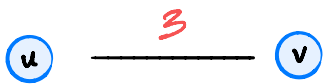
Today

How do we find shortest paths in weighted graphs?

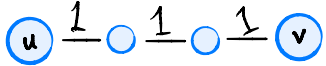
Idea #0

- Does BFS work?
  - **No**, not really. Only if all weights are the same.
- Can we “convert” a weighted graph to an unweighted one?

Idea #0



Idea #0



Idea #0

- Step 1: “Convert” weighted graph to unweighted one with dummy nodes.
- Step 2: Call BFS on this new graph.

Idea #0

- **Very inefficient** for large weights.
- What if edge weights are floats, or negative?

Ideas #1 and #2

- We'll look at two algorithms: **Bellman-Ford** and **Dijkstra's**.

  INPUT: weighted graph, source vertex s.
  OUTPUT: shortest paths from s to every other node.

- Both work by:
  - keeping track of shortest known path (**estimates**).
  - iteratively **updating** these until they're correct.

Shortest Path Estimates

- B-F and Dijkstra's keep track of the shortest paths found *so far*; we call these the **estimated shortest paths**.

- For each node u, remember u's:
  - predecessor in estimated shortest path;
  - distance from source s in estimated shortest path.

- **Key**: estimated distance will always be ≥ actual distance.

Updates

- Both algorithms work by iteratively updating their estimates.
- On each iteration, consider a new edge (u, v). Ask: is the best known shortest path from

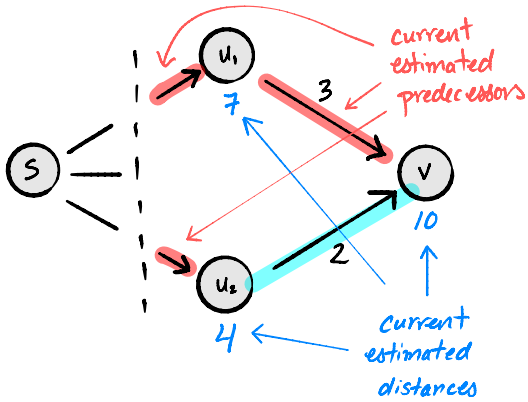
  $$\text{source} \to \cdots \to u \to v$$

  shorter than the best known shortest path from

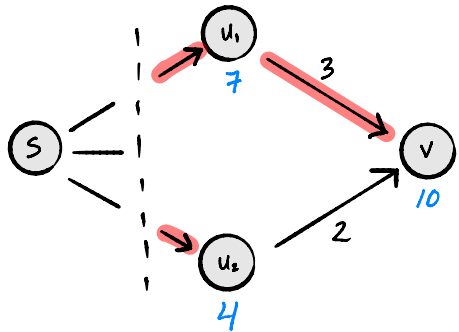
  $$\text{source} \to \cdots \to \texttt{predecessor[v]} \to v?$$

- If it is, we have discovered a shorter path to v.

Example: Updating (u_2, v):



Example: Updating (u_2, v):

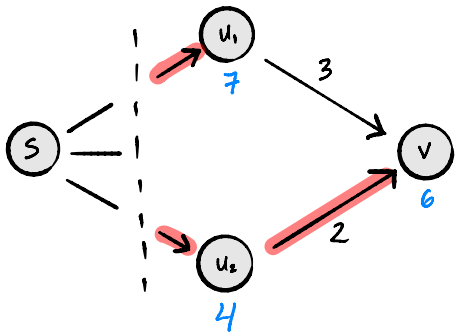


estimated length of $s \to u_2 \to v$

$$= (\text{estimated length of } s \to u_2) + \omega(u_2, v)$$

$$= 4 + 2 = 6 < 10$$

Example: After Updating (u_2, v):



A shorter path has been found.

Updating, in Code

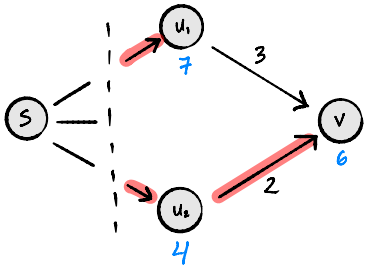
- Let:
  - `est` be a dictionary of estimated shortest path distances.
  - `predecessor` be a dictionary of estimated shortest path predecessors.
  - `weights` be a function which returns edge weights.

Updating, in Code

```
def update(u, v, weights, est, predecessor):
    """Update edge (u,v)."""
    if est[v] > est[u] + weights(u,v):
        est[v] = est[u] + weights(u,v)
        predecessor[v] = u
        return True
    else:
        return False
```

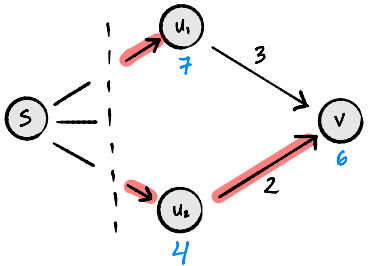
- Time complexity: $\Theta(1)$

When does an update discover a shortest path?



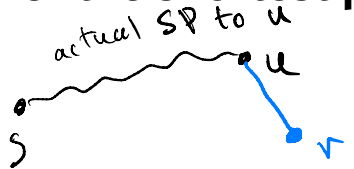
- Suppose updating (u_2, v) finds a shorter path to v.
- True or False: the actual shortest path must go through u_2.

When does an update discover a shortest path?



- Suppose updating (u_2, v) finds a shorter path to v.
- True or False: the actual shortest path must go through u_2.
- **False**: we might later discover a better path to u_1.

When does an update discover the shortest path?



- Let (u, v) be an edge.
- Suppose:
  - the actual shortest path to u has been found;
  - the actual shortest path to v goes through (u, v).
- Then after updating (u, v), the estimated shortest path to v is correct.

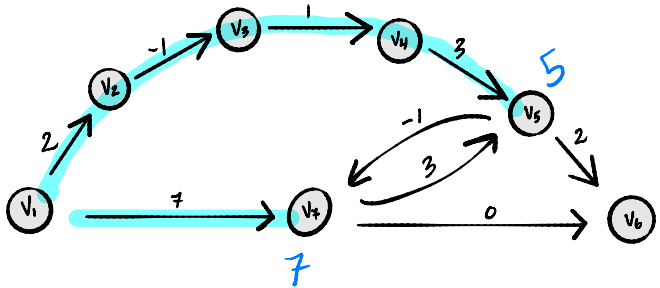
CS-GY 6033
Design and Analysis of Algorithms I

Lecture 8 | Part 2

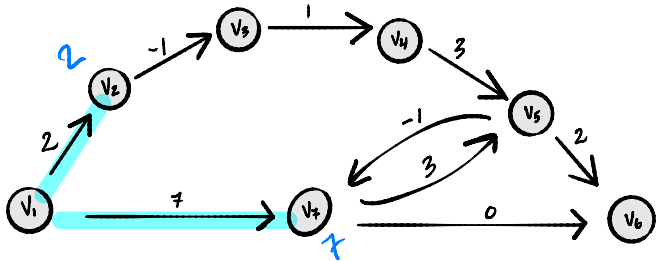
Bellman-Ford

Intuition

- Shortest paths that have many edges are “harder” to discover.
  - May require many updates.
- Shortest paths that have few edges are “easier” to discover.
- Once we’ve discovered all of the shortest paths with few edges, it makes it easier to discover the shortest paths with more edges.

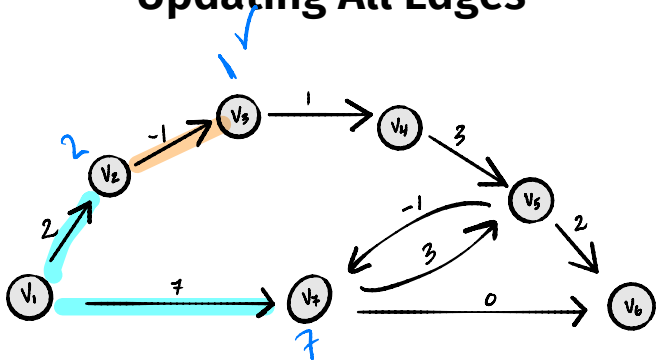

v1
v2
v3
v4
v5
v6
v7
2
-1
1
3
5
-1
3
2
7
0
7

Updating All Edges



- Suppose we update all of the edges, one by one.
- Then all nodes whose shortest path from *s* has only **one** edge are **guaranteed** to be estimated correctly.

Updating All Edges



- Suppose we update all of the edges again.
- Then all nodes whose shortest path from s has at most **two** edges are **guaranteed** to be estimated correctly.

Loop Invariant

- One iteration: update all edges in arbitrary order.
- Loop invariant: After α iterations, all nodes whose shortest path from s has $\leq \alpha$ edges are guaranteed to be estimated correctly.

The Bellman-Ford Algorithm

```
def bellman_ford(graph, weights, source):
    """Assume graph is directed."""
    est = {node: float('inf') for node in graph.nodes}
    est[source] = 0
    predecessor = {node: None for node in graph.nodes}

    for i in range(?):
        for (u, v) in graph.edges:
            update(u, v, weights, est, predecessor)

    return est, predecessor
```

|V| - 1

Bellman-Ford

- Claim: each node must have a shortest path which is simple[1].
- The most edges a simple path can have is $|V| - 1$
- Idea of Bellman-Ford: iteratively update all edges, repeat $|V| - 1$ times.

[1] Edge case: cycles of weight zero.

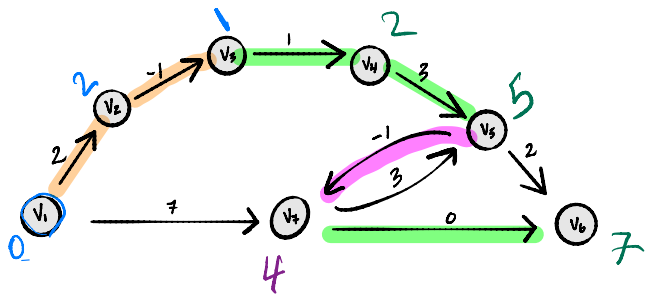
The Bellman-Ford Algorithm

```
def bellman_ford(graph, weights, source):
    """Assume graph is directed."""
    est = {node: float('inf') for node in graph.nodes}
    est[source] = 0
    predecessor = {node: None for node in graph.nodes}

    for i in range(len(graph.nodes) - 1):
        for (u, v) in graph.edges:
            update(u, v, weights, est, predecessor)

    return est, predecessor
```

Example

Suppose `graph.edges` returns edges in following order:

$$(v_3, v_4), (v_1, v_2), (v_2, v_3), (v_7, v_6), (v_5, v_7),$$
$$(v_7, v_5), (v_4, v_5), (v_5, v_6), (v_1, v_7)$$



Time Complexity

```
def bellman_ford(graph, weights, source):
    """Assume graph is directed."""
    est = {node: float('inf') for node in graph.nodes}
    est[source] = 0
    predecessor = {node: None for node in graph.nodes}

    for i in range(len(graph.nodes) - 1):
        for (u, v) in graph.edges:
            update(u, v, weights, est, predecessor)

    return est, predecessor
```

- Setup: $\Theta(V)$ time
- Each update takes $\Theta(1)$ time
- There are exactly $\Theta(V \cdot E)$ updates
- Total time complexity: $\Theta(V \cdot E + V)$

CS-GY 6033
Design and Analysis of Algorithms I

Lecture 8 | Part 3

Early Stopping and Negative Cycles

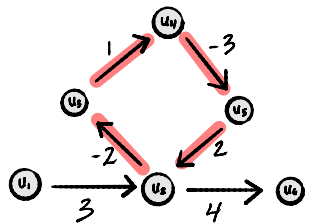
Early Stopping

- B-F may not need to run for $|V| - 1$ iterations.
- If no predecessors change, we can break:

```
def bellman_ford(graph, weights, source):
    """Early stopping version."""
    est = {node: float('inf') for node in graph.nodes}
    est[source] = 0
    predecessor = {node: None for node in graph.nodes}

    for i in range(len(graph.nodes) - 1):
        any_changes = False
        for (u, v) in graph.edges:
            changed = update(u, v, weights, est, predecessor)
            any_changes = changed or any_changes
        if not any_changes:
            break
    return est, predecessor
```

Negative Cycles

- A **negative cycle** is a cycle whose total edge weight is negative:



- If a graph has a negative cycle, (some) shortest paths are **not well defined**.

Detecting Negative Cycles

- If graph **does not have** negative cycles, estimated distances eventually stop changing (after at most $|V| - 1$ iterations).
- If graph **has** negative cycles, estimated distances always decrease.
- To detect them: run a $|V|$th iteration; if distances change, a negative cycle exists.

Detecting Negative Cycles

```
def bellman_ford(graph, weights, source):
    """Early stopping version, detects negative cycles."""
    est = {node: float('inf') for node in graph.nodes}
    est[source] = 0
    predecessor = {node: None for node in graph.nodes}

    for i in range(len(graph.nodes)):
        any_changes = False
        for (u, v) in graph.edges:
            changed = update(u, v, weights, est, predecessor)
            any_changes = changed or any_changes
        if not any_changes:
            break
    # this will be True if negative cycles exist
    contains_negative_cycles = any_changes
    return est, predecessor, contains_negative_cycles
```

CS-GY 6033
Design and Analysis of Algorithms I

Lecture 8 | Part 4

Dijkstra's Algorithm

Shortest Path Algorithms

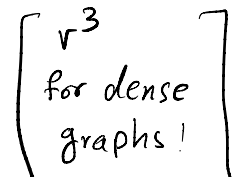
- **Bellman-Ford** and **Dijkstra's** are shortest path algorithms:

  INPUT: weighted graph, source vertex s.
  OUTPUT: shortest paths from s to every other node.

- Both work by:
  - keeping estimates of shortest path distances;
  - iteratively **updating** estimates until they're correct.

Shortest Path Algorithms

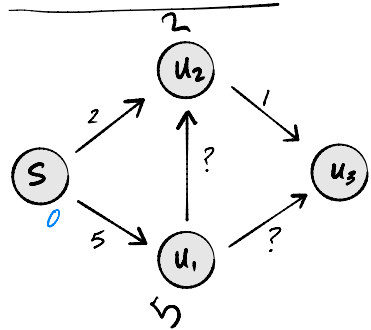
- We saw Bellman-Ford last time; takes time $\Theta(VE + V)$
- Dijkstra's will be faster, but can't handle negative weights.



Dijkstra's Algorithm

- On every iteration, Bellman-Ford updates all edges – many don't need to be updated.

- If we **assume** all edge weights are positive, we can rule out some paths immediately:



Dijkstra's Idea

- Keep track of set C of "correct" nodes.
  - Nodes whose distance estimate is correct.
- At every step, add node outside of C with smallest estimated distance; update only its neighbors.
- A "greedy" algorithm.

Outline of Dijkstra's Algorithm

```
def dijkstra(graph, weights, source):
    est = {node: float('inf') for node in graph.nodes}
    est[source] = 0
    pred = {node: None for node in graph.nodes}

    # empty set
    C = set()

    # while there are nodes still outside of C
        # find node u outside of C with smallest
        # estimated distance
        C.add(u)
        for v in graph.neighbors(u):
            update(u, v, weights, est, pred)

    return est, pred
```

Example

$C = \{\ \}$ $V \setminus C = \{$ every thing $\}$

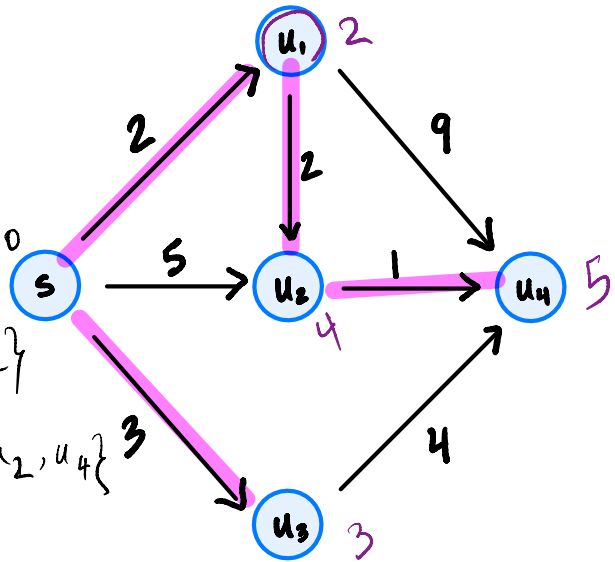
$C = \{s\}$

$C = \{s, u_1\}$

$C = \{s, u_1, u_3\}$

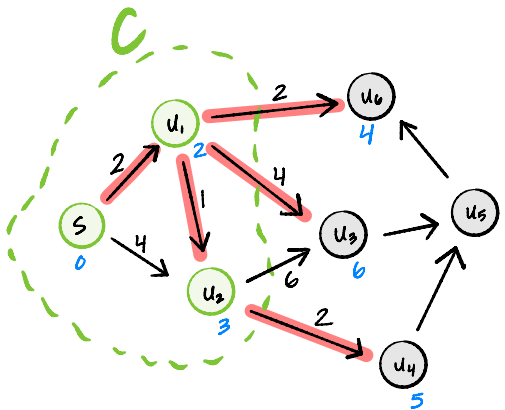
$C = \{s, u_1, u_3, u_2\}$

$C = \{s, u_1, u_3, u_2, u_4\}$



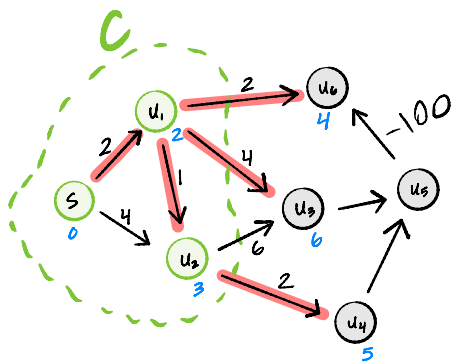
Proof Idea

- Claim: at beginning of any iteration of Dijkstra's, if u is node $\notin C$ with smallest estimated distance, the shortest path to u has been correctly discovered.



Proof Idea

- Let *u* be node outside of *C* for which `est[u]` is smallest.
- We've discovered a path from *s* to *u* of length `est[u]`.
- Any path from *s* to *u* has to exit *C* somewhere.
- Any path from *s* to *u* will cost at least `est[u]` just to exit *C*.



Exercise

Why do the edge weights need to be positive? Come up with a simple example graph with some negative edge weights where Dijkstra's fails to compute the correct shortest path.

CS-GY 6033
Design and Analysis of Algorithms I

Lecture 8 | Part 5

Implementation

Outline of Dijkstra's Algorithm

```
def dijkstra(graph, weights, source):
    est = {node: float('inf') for node in graph.nodes}
    est[source] = 0
    pred = {node: None for node in graph.nodes}

    # empty set
    C = set()

    # while there are nodes still outside of C
        # find node u outside of C with smallest
        # estimated distance
        C.add(u)
        for v in graph.neighbors(u):
            update(u, v, weights, est, pred)

    return est, pred
```

Dijkstra's Algorithm: Naïve Implementation

```
1  def dijkstra(graph, weights, source):
2      est = {node: float('inf') for node in graph.nodes}
3      est[source] = 0
4      pred = {node: None for node in graph.nodes}
5
6      outside = set(graph.nodes)
7
8      while outside:
9          # find smallest with linear search
10         u = min(outside, key=est)
11         outside.remove(u)
12         for v in graph.neighbors(u):
13             update(u, v, weights, est, pred)
14
15     return est, pred
```

- Time complexity: _____

Priority Queues

- A **priority queue** allows us to store (key, value) pairs, efficiently return key with lowest value.

- Suppose we have a priority queue class:
  - `PriorityQueue(priorities)` will create a priority queue from a dictionary whose values are priorities.
  - The `.extract_min()` method removes and returns key with smallest value.
  - The `.change_priority(key, value)` method changes key's value.

Example

```
>>> pq = PriorityQueue({
    'w': 5,
    'x': 4,
    'y': 1,
    'z': 3
})
>>> pq.extract_min()
'y'
>>> pq.change_priority('w', 2)
>>> pq.extract_min()
```

Dijkstra's Algorithm: Priority Queue

```
def dijkstra(graph, weights, source):
    est = {node: float('inf') for node in graph.nodes}
    est[source] = 0
    pred = {node: None for node in graph.nodes}

    priority_queue = PriorityQueue(est)
    while priority_queue:
        u = priority_queue.extract_min()
        for v in graph.neighbors(u):
            changed = update(u, v, weights, est, pred)
            if changed:
                priority_queue.change_priority(v, est[v])

    return est, pred
```

Heaps

- A priority queue can be implemented using a **heap**.
- If a **binary min-heap** is used:
  - `PriorityQueue(est)` takes $\Theta(V)$ time.
  - `.extract_min()` takes $O(\log V)$ time.
  - `.change_priority()` takes $O(\log V)$ time.

Time Complexity Using Min Heap

```
def dijkstra(graph, weights, source):
    est = {node: float('inf') for node in graph.nodes}
    est[source] = 0
    pred = {node: None for node in graph.nodes}

    priority_queue = PriorityQueue(est)
    while priority_queue:
        u = priority_queue.extract_min()
        for v in graph.neighbors(u):
            changed = update(u, v, weights, est, pred)
            if changed:
                priority_queue.change_priority(v, est[v])

    return est, pred
```

$O(\log |V|)$

$O(\log |V|)$

- Time complexity: $O(V \log V + E \log V)$

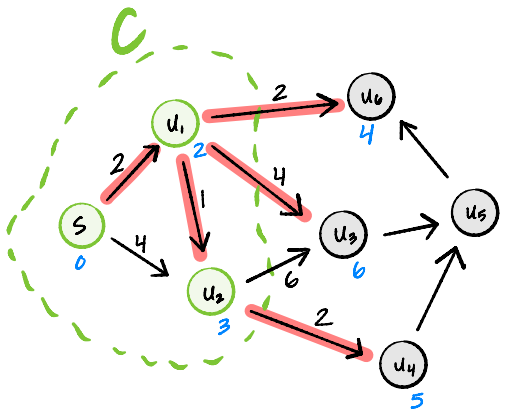
CS-GY 6033
Design and Analysis of Algorithms I

Lecture 8 | Part 6

Proof

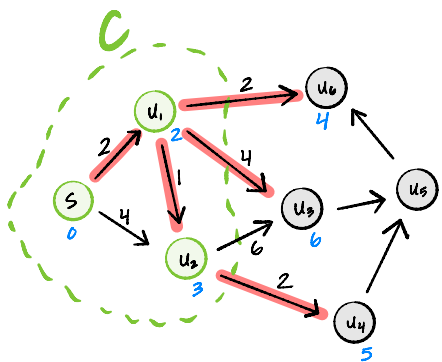
Proof Idea

- Claim: at beginning of any iteration of Dijkstra's, if u is node $\notin C$ with smallest estimated distance, the shortest path to u has been correctly discovered.



Proof Idea

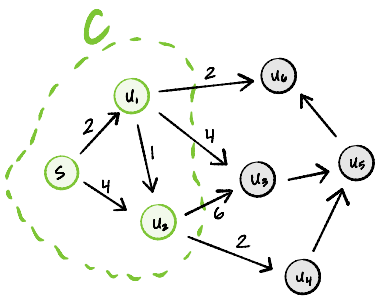
- Let *u* be node outside of *C* for which `est[u]` is smallest.
- We've discovered a path from *s* to *u* of length `est[u]`.
- Any path from *s* to *u* has to exit *C* somewhere.
- Any path from *s* to *u* will cost at least `est[u]` just to exit *C*.



Exit Paths

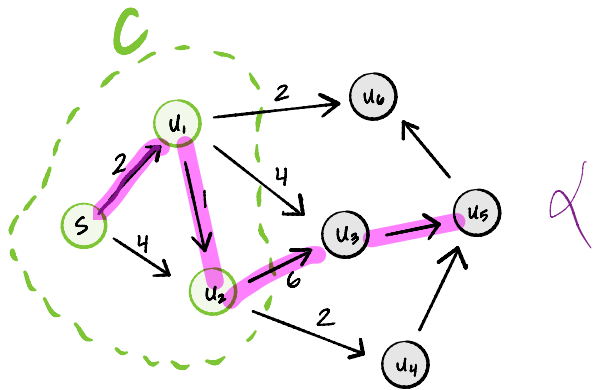
- An **exit path from s through C** is a path for which:
  - the first node is s;
  - the last node (a.k.a., the **exit node**) is **not** in C;
  - all other nodes **are** in C.

- Example:



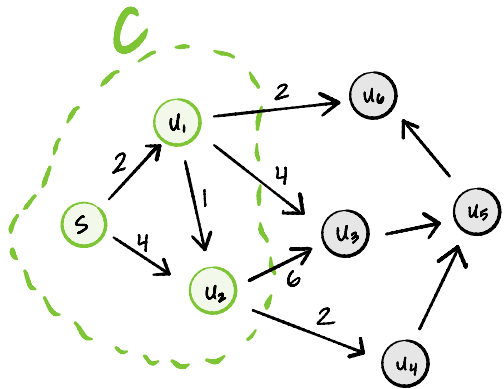
Exit Paths

- True or False: this is an exit path from s through C.



Exit Paths

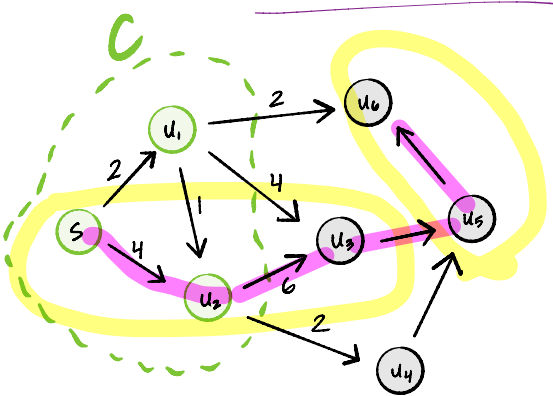
- True or False: this is an exit path from s through C.



Path Decomposition

- Any path from s to a node u outside of C can be broken into two parts:

(an exit path from s) + (path from exit node to u)



Path Decomposition

- Consider any path from s to $u \notin C$.
- Suppose e is the path's exit node.
- We have:

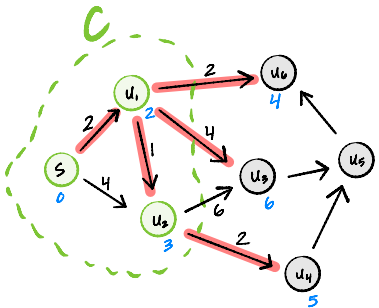
(length of the path)
= (length of exit path to e) + (length of path from e to u)
≥ (length of shortest exit path to e) + (length of path from e to u)

- Since edge weights are positive, all path lengths ≥ 0:

≥ (length of shortest exit path to e) + 0

Shortest Exit Paths

- Example: What is the shortest exit path with exit node u_3?



- If u is outside of C, then the length of the shortest exit path with exit node e is `est[e]`.

Proof Idea

- Suppose *u* is a node outside of *C* for which `est[u]` is smallest.
- Consider any path from *s* to *u*, and let *e* be the path's exit node.
- We have:

(length of this path from *s* to *u*)
≥ (length of shortest exit path to *e*) + 0
= `est[e]`
≥ `est[u]`

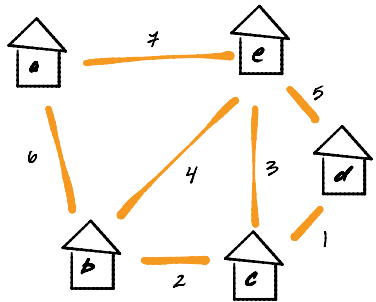
- That is, any path from *s* to *u* has length ≥ `est[u]`.
- We've already found one with length `est[u]`; this proves that it is the shortest.

CS-GY 6033
Design and Analysis of Algorithms I

Lecture 8 | Part 7

Minimum Spanning Trees

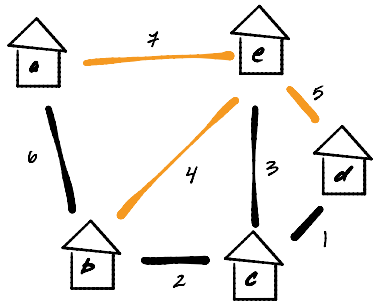
Today's Problem



- Choose a set of dirt roads to pave so that:
  - can get between any two buildings only on paved roads;
  - total cost is minimized.

- Solution: compute a **minimum spanning tree**.

Today's Problem



- Choose a set of dirt roads to pave so that:
  - can get between any two buildings only on paved roads;
  - total cost is minimized.

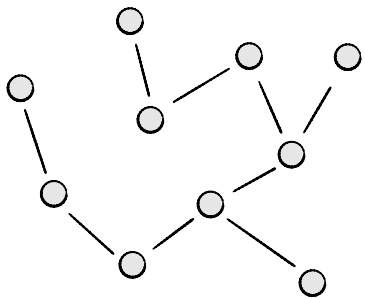
- Solution: compute a **minimum spanning tree**.

Trees

An undirected graph $T = (V, E)$ is a **tree** if

- it is connected; and
- it is acyclic.

Example: a **tree**.

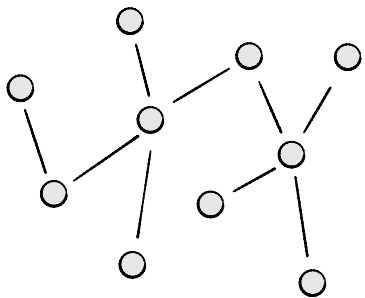


Trees

An undirected graph $T = (V, E)$ is a **tree** if

- it is connected; and
- it is acyclic.

Example: a **tree**.

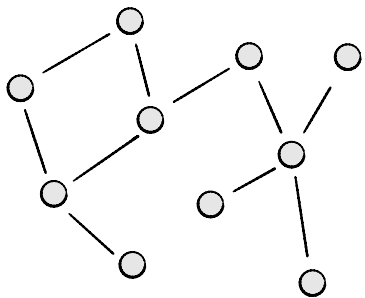


Trees

An undirected graph $T = (V, E)$ is a **tree** if

- it is connected; and
- it is acyclic.

Example: **not** a tree.

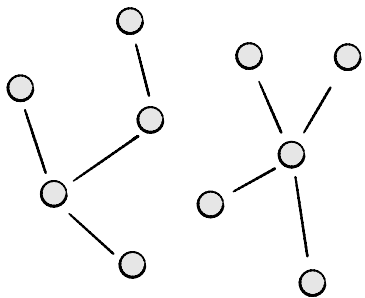


Trees

An undirected graph $T = (V, E)$ is a **tree** if

- it is connected; and
- it is acyclic.

Example: **not** a tree.

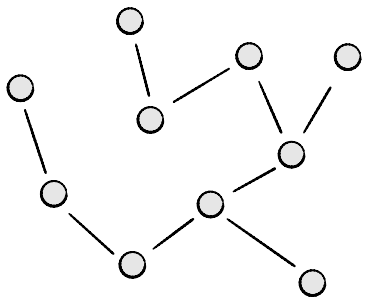


Trees: Equivalent Definition

An undirected graph $T = (V, E)$ is a **tree** if

- it is connected; and
- $|E| = |V| - 1$.

Example: a **tree**.

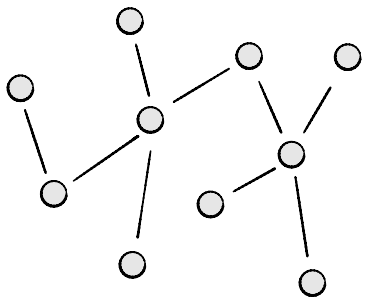


Trees: Equivalent Definition

An undirected graph $T = (V, E)$ is a **tree** if

- it is connected; and
- $|E| = |V| - 1$.

Example: a **tree**.



Trees: Equivalent Definition

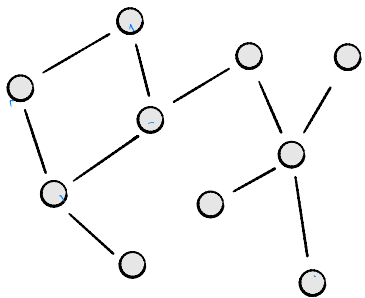
An undirected graph $T = (V, E)$ is a **tree** if

- it is connected; and
- $|E| = |V| - 1$.

$|E| = 10$

$|V| = 10$

Example: **not** a tree.



Trees: Equivalent Definition

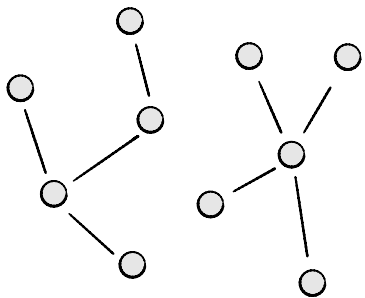
An undirected graph $T = (V, E)$ is a **tree** if

- it is connected; and
- $|E| = |V| - 1$.

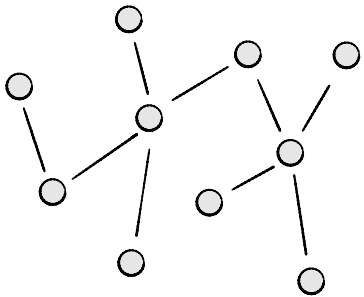
$|V| = 10$

$|E| = 8$

Example: **not** a tree.



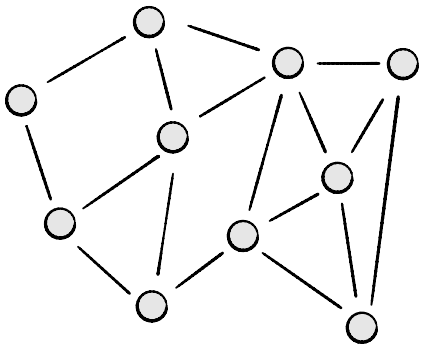
Tree Properties



- There is a unique simple path between any two nodes in a tree.
- Adding a new edge to a tree creates a cycle (no longer a tree).
- Removing an edge from a tree disconnects it (no longer a tree).

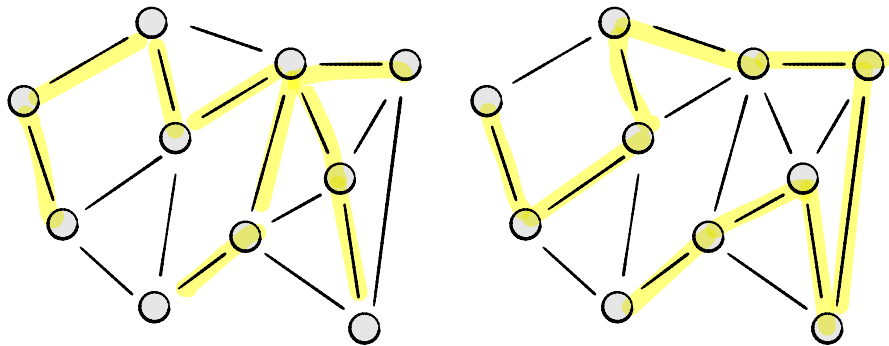
Spanning Trees

Let $G = (V, E)$ be a **connected** graph. A **spanning tree** of G is a tree $T = (V, E_T)$ with the same nodes as G, and a subset of G's edges.



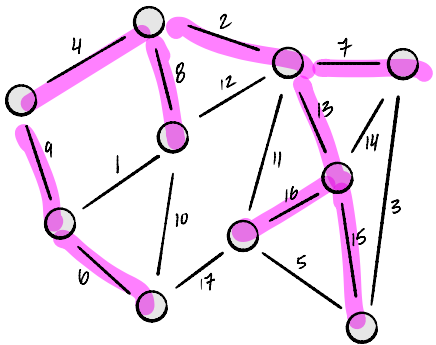
Many Spanning Trees

The same graph can have many spanning trees.



Spanning Tree Cost

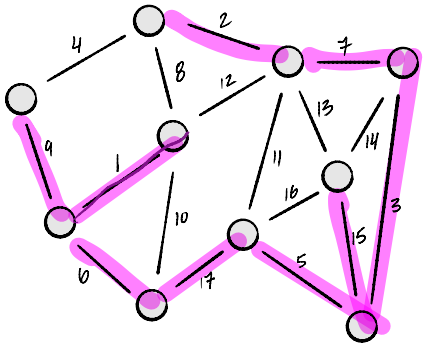
If $G = (V, E, \omega)$ is a weighted undirected graph, the **cost** (or **weight**) of a spanning tree is the total weight of the edges in the spanning tree.



Cost: $4 + 2 + 7 + 13 + 15 + 16 + 8 + 9 + 6$

Spanning Tree Cost

Different spanning trees of the same graph can have different costs.



Cost: 2

Minimum Spanning Tree

- The **minimum spanning tree** problem is as follows:
  - GIVEN: A weighted, undirected graph $G = (V, E, \omega)$.
  - COMPUTE: a spanning tree of G with minimum cost (i.e., minimum total edge weight).

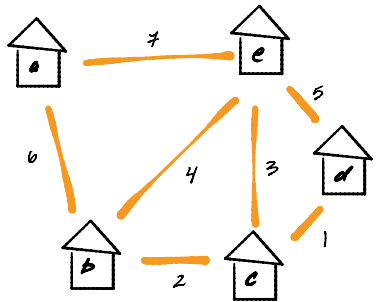
- For a given graph, the MST may not be unique.

Exercise

Suppose the edges of a graph $G = (V, e, \omega)$ all have the same weight. How can we compute an MST of the graph?

BFS tree

Today's Problem



- Choose a set of dirt roads to pave so that:
  - can get between any two buildings only on paved roads;
  - total cost is minimized.

- Solution: compute a **minimum spanning tree**.

MSTs in Data Science?

- Do we need to find MSTs in data science?
- Actually, yes! (Next lecture)

CS-GY 6033
Design and Analysis of Algorithms I

Lecture 8 | Part 8

Prim's Algorithm

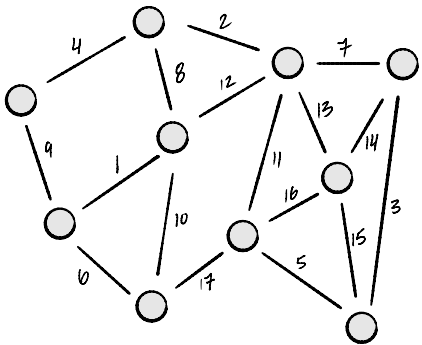
Building MSTs

- How do we build a MST efficiently?
- We'll adopt a **greedy** approach.
  - Build a tree edge-by-edge.
  - At every step, doing what looks best at the moment.
- This strategy isn't guaranteed to work in all of life's situations, but it works for building MSTs.

Two Greedy Approaches

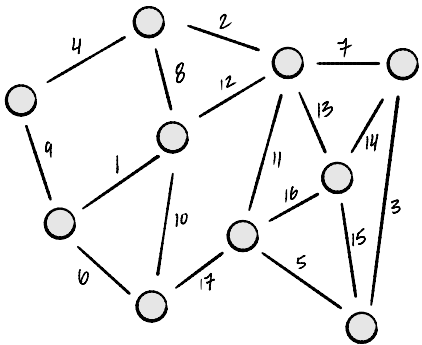
- We'll look at two greedy algorithms:
  - Today: Prim's Algorithm
  - Next time: Kruskal's Algorithm
- Differ in the order in which edges are added to tree.
- Also differ in time complexity.

Prim's Algorithm, Informally



- Start by picking any node to add to "tree", *T*.
- While *T* is not a spanning tree, greedily add **lightest** edge from a node in *T* to a node not in *T*.
  - "lightest" = edge of smallest weight

Prim's Algorithm, Informally



- Start by picking any node to add to "tree", *T*.
- While *T* is not a spanning tree, greedily add **lightest** edge from a node in *T* to a node not in *T*.
  - "lightest" = edge of smallest weight
- **Is this guaranteed to work?** Yes, as we'll see.

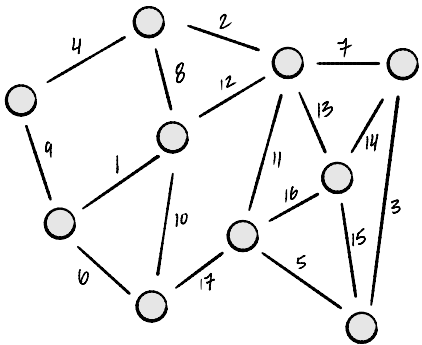
Prim's Algorithm, Equivalently

- For each node u, store:
  - estimated cost of adding node to tree;
  - estimated "predecessor" v in the tree.

- At each step,
  - Find node with smallest estimated cost.
  - Add to tree T by including edge with estimated "predecessor".
  - Update cost of neighbors.

- Same as adding lightest edge from T to outside T at every step!

Prim's Algorithm, Equivalently



- While T is not a tree:
  - find the node $u \notin T$ with smallest cost
  - add the edge between u and its estimated "predecessor" to T
  - update estimated cost/pred. of u's neighbors which aren't already in tree.

Recall: Priority Queues

- How do we efficiently find node with smallest cost?
- Priority Queues:
  - `PriorityQueue(priorities)`: creates priority queue from dictionary whose values are priorities.
  - `.extract_min()`: removes and returns key with smallest value.
  - `.decrease_priority(key, value)`: changes key's value.
- We'll use a priority queue to hold nodes not yet added to tree.

```python
def prim(graph, weight):
    tree = UndirectedGraph()

    estimated_predecessor = {node: None for node in graph.nodes}
    cost = {node: float('inf') for node in graph.nodes}
    priority_queue = PriorityQueue(cost)

    while priority_queue:
        u = priority_queue.extract_min()
        if estimated_predecessor[u] is not None:
            tree.add_edge(estimated_predecessor[u], u)
        for v in graph.neighbors(u):
            if weight(u, v) < cost[v] and v not in tree.nodes:
                priority_queue.decrease_priority(v, weight(u, v))
                cost[v] = weight(u, v)
                estimated_predecessor[v] = u
    return tree
```

Prim and Dijkstra

- This is a lot like Dijkstra's Algorithm for s.p.d.!
- Both: at each step, extract node with smallest cost, update its edges. (Prim: only those edges to nodes not in tree).
- Dijkstra update of (u, v):

```
cost[v] = min(cost[v], cost[u] + weight(u, v))
```

- Prim update of (u, v):

```
cost[v] = min(cost[v], weight(u, v))
```

CS-GY 6033
Design and Analysis of Algorithms I

Lecture 8 | Part 9

Time Complexity

Time Complexity

- A priority queue can be implemented using a **heap**.

- If a **binary min-heap** is used:
  - `PriorityQueue(est)` takes $\Theta(V)$ time.
  - `.extract_min()` takes $O(\log V)$ time.
  - `.decrease_priority()` takes $O(\log V)$ time.

Time Complexity

```
def prim(graph, weight):
    tree = UndirectedGraph()

    estimated_predecessor = {node: None for node in graph.nodes}
    cost = {node: float('inf') for node in graph.nodes}
    priority_queue = PriorityQueue(cost)

    while priority_queue:
        u = priority_queue.extract_min()
        if estimated_predecessor[u] is not None:
            tree.add_edge(estimated_predecessor[u], u)
        for v in graph.neighbors(u):
            if weight(u, v) < cost[v] and v not in tree.nodes:
                priority_queue.decrease_priority(v, weight(u, v))
                cost[v] = weight(u, v)
                estimated_predecessor[v] = u
    return tree
```

Time Complexity

- Using a **binary heap**...
- Overall: $\Theta(V \log V + E \log V)$.
- Since graph is assumed connected, $E = \Omega(V)$.
- So this simplifies to $\Theta(E \log V)$.

Fibonacci Heaps

- A priority queue can be implemented using a **heap**.
- If a **Fibonacci min-heap** is used:
  - `PriorityQueue(est)` takes $\Theta(V)$ time.
  - `.extract_min()` takes $\Theta(\log V)$ time[2].
  - `.decrease_priority()` takes $O(1)$ time.

[2] Amortized.

Time Complexity

```
def prim(graph, weight):
    tree = UndirectedGraph()

    estimated_predecessor = {node: None for node in graph.nodes}
    cost = {node: float('inf') for node in graph.nodes}
    priority_queue = PriorityQueue(cost)

    while priority_queue:
        u = priority_queue.extract_min()
        if estimated_predecessor[u] is not None:
            tree.add_edge(estimated_predecessor[u], u)
        for v in graph.neighbors(u):
            if weight(u, v) < cost[v] and v not in tree.nodes:
                priority_queue.decrease_priority(v, weight(u, v))
                cost[v] = weight(u, v)
                estimated_predecessor[v] = u
    return tree
```

Time Complexity

- Using a **Fibonacci heap**...
- Overall: $\Theta(V \log V + E)$.

Fibonacci vs. Binary Heaps

- Using Fibonacci heaps improves time complexity when graph is dense.
- E.g., if $E = \Theta(V^2)$:
  - Prim's with Fibonacci: $\Theta(E) = \Theta(V^2)$
  - Prim's with binary: $\Theta(E \log E) = \Theta(V^2 \log V)$.
- But Fibonacci heaps are **hard** to implement; have large constants.
- Binary heaps used more in practice despite complexity.

CS-GY 6033
Design and Analysis of Algorithms I

Lecture 8 | Part 10

Correctness of Prim's Algorithm

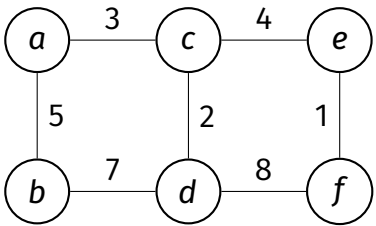
Being Greedy

- At every step, we add the lightest edge.
- Is this “safe”?

Being Greedy

- At every step, we add the lightest edge.
- Is this “safe”?
- Yes! This is guaranteed to find an MST.

Promising Subtrees

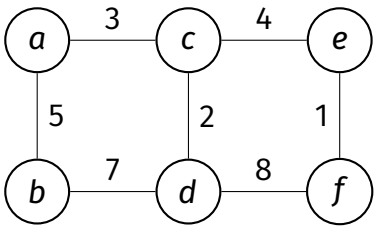


- Let $G = (V, E, \omega)$ be a weighted graph.
- A subgraph $T' = (V', E')$ is **promising** if it is "part" of some MST.
  - That is, it is an "MST in progress"
  - Not necessarily a tree!
- That is, there exists an MST $T = (V, E_{\text{mst}})$ such that $E' \subset E_{\text{mst}}$.
- Hint: a "promising subtree" where $V' = V$ is an MST!

Main Idea

Prim's starts with a promising subtree T. At each step, adds lightest edge from a node within T to a node outside of T.

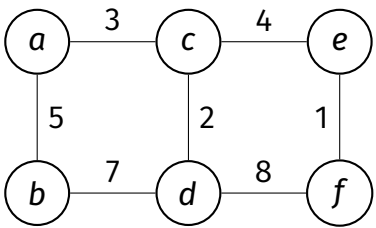
We'll show each new edge results in a larger promising subtree. Eventually the promising subtree becomes a full MST.



Claim

- Let $G = (V, E, \omega)$ be a weighted graph.
- Suppose $T' = (V', E')$ is a promising subtree for an MST of G.
- Let $e = (u, v)$ be a **lightest edge** from a node in T' to a node outside of T'. (Prim).
- Then adding (u, v) to T' results in another **promising subtree**.

Proof



- Suppose T_{mst} is an MST that includes T'.
- If T_{mst} includes e, we're done: $T' + e$ is promising.
- If it doesn't include e, it must have an edge f that connects T' to rest of the graph.
- Swap f with e in T_{mst}. The result is a tree, and it must be a MST since $\omega(e) \leq \omega(f)$.
- So there is an MST that contains $T' + e$.

CS-GY 6033
Design and Analysis of Algorithms I

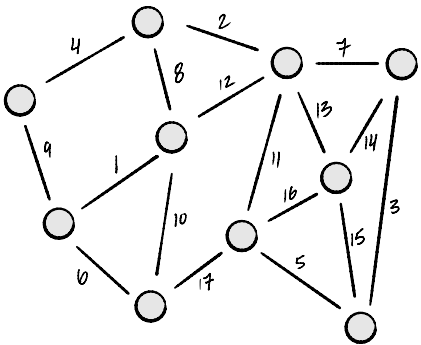
Lecture 8 | Part 11

Kruskal's Algorithm

Two Greedy Approaches

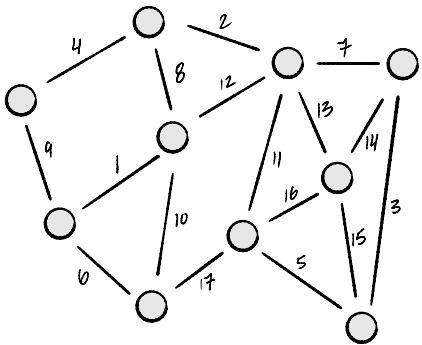
- We'll look at two greedy algorithms:
  - Prim's Algorithm
  - Kruskal's Algorithm
- Differ in the order in which edges are added to tree.
- Also differ in time complexity.

Prim's Algorithm, Informally



- Start by picking any node to add to "tree", *T*.
- While *T* is not a spanning tree, greedily add **lightest** edge from a node in *T* to a node not in *T*.
  - "lightest" = edge of smallest weight

Kruskal's Algorithm, Informally



- Start with empty forest: $T = (V, E_{\text{mst}})$, where $E_{\text{mst}} = \emptyset$.
- Loop through edges in increasing order of weight.
  - If edge does not create a cycle in T, add it to T.
  - If T is a spanning tree, break.

Being Greedy

- Prim: add the **node** with smallest estimated cost and update neighbors.
  - Works locally, “grows” a connected tree.

- Kruskal: add the **edge** with smallest weight.
  - As long as it doesn’t make a cycle.
  - Edge can be anywhere in graph.

Kruskal's Algorithm (Pseudocode)

```
def kruskal(graph, weights):
    mst = UndirectedGraph()

    # sort edges in ascending order by weight
    sorted_edges = sorted(graph.edges, key=weights)

    for (u, v) in sorted_edges:
        # if u and v are not already connected
        if ...:
            mst.add_edge(u, v)

            # (optional) if mst is now a spanning tree, break
            if len(mst.edges) == len(graph.nodes) - 1:
                break

    return mst
```

Checking for Connectivity

- Each iteration: check if u and v are connected in $T = (V, E_{\text{mst}})$.
- We *could* do a DFS/BFS on each iteration...
  - $\Theta(V + E_{\text{mst}}) = \Theta(V)$ each time.
  - **Expensive**!
- Remember:
  - If you're computing something once, use a fast algorithm.
  - If you're computing it repeatedly, consider a **data structure**.

Disjoint Set Forests

- Represent a collection of disjoint sets.

$$\{\{1, 5, 6\}, \{2, 3\}, \{0\}, \{4\}\}$$

- `.union(x, y)`: Union the sets containing x and y.
- `.in_same_set(x, y)`: Return `True`/`False` if x and y are in the same set.[3]

[3] Usually implemented as a `.find(x)` method returning representative of set containing `x`.

Example

```
>>> # create a DSF with {{0}, {1}, {2}, {3}, {4}, {5}}
>>> dsf = DisjointSetForest([0, 1, 2, 3, 4, 5])
>>> dsf.union(0, 3)
>>> dsf.union(1, 4)
>>> dsf.union(3, 1)
>>> dsf.union(2, 5)
>>> # dsf now represents {{0, 1, 3, 4}, {2, 5}}
>>> dsf.in_same_set(0, 3)
True
>>> dsf.in_same_set(0, 2)
False
```

Disjoint Set Forests

- Operations take $\Theta(\alpha(n))$ time, where n is number of objects in collection.
- $\alpha(n)$ is the **inverse Ackermann function**.
- It grows very, **very** slowly.
- Essentially constant time.

Disjoint Set Forests

- Can be used to keep track of CCs of a **dynamic graph**.
- Nodes of CCs are disjoint sets.
  - Add an edge (u, v): `.union(u, v)`
  - Check if u and v are connected: `.in_same_set(u, v)`
- To check if u, v are already connected:
  - BFS/DFS: $\Theta(V)$ each time.
  - DSF: $\Theta(\alpha(V))$ each time (essentially $\Theta(1)$).

Kruskal's Algorithm

```
def kruskal(graph, weights):
    mst = UndirectedGraph()

    # place each node in its own disjoint set
    components = DisjointSetForest(graph.nodes)

    # sort edges in ascending order by weight
    sorted_edges = sorted(graph.edges, key=weights)

    for (u, v) in sorted_edges:
        if not components.in_same_set(u, v):
            mst.add_edge(u, v)
            components.union(u, v)

            # (optional) if mst is now a spanning tree, break
            if len(mst.edges) == len(graph.nodes) - 1:
                break

    return mst
```

Time Complexity

```
def kruskal(graph, weights):
    mst = UndirectedGraph()

    # place each node in its own disjoint set
    components = DisjointSetForest(graph.nodes)

    # sort edges in ascending order by weight
    sorted_edges = sorted(graph.edges, key=weights)

    for (u, v) in sorted_edges:
        if not components.in_same_set(u, v):
            mst.add_edge(u, v)
            components.union(u, v)

            # (optional) if mst is now a spanning tree, break
            if len(mst.edges) == len(graph.nodes) - 1:
                break

    return mst
```

Time Complexity

- Assume graph is connected. Then $E = \Omega(V)$.
- Kruskal's takes $\Theta(E \log E) = \Theta(E \log V)$ time.
  - Dominated by sorting the edges.
- Note: if graph disconnected, Kruskal's produces a **minimum spanning forest**.

CS-GY 6033
Design and Analysis of Algorithms I

Lecture 8 | Part 12

Kruskal v. Prim

Kruskal v. Prim

- Both algorithms for computing MSTs.
- Which is “better”?
- There’s no clear winner.

Time Complexity

- Prim:
  - Binary heap: $\Theta(V \log V + E \log V)$
  - Fibonacci heap: $\Theta(V \log V + E)$
- Kruskal: $\Theta(E \log V)$
- If the graph is dense, $E = \Theta(V^2)$, and Prim's with Fibonacci heap "wins".
  - $\Theta(V^2)$ versus $\Theta(V^2 \log V)$.

Not so fast...

- Fibonacci heaps are hard to implement, high overhead.
- Prim's will be faster for very large dense graphs.
- But Kruskal's may be faster for smaller dense graphs.
- The right choice depends on your application.

Main Idea

Asymptotic time complexity isn't everything. For small inputs, the "inefficient" algorithm may beat the "efficient" one. There's also ease of implementation to consider.

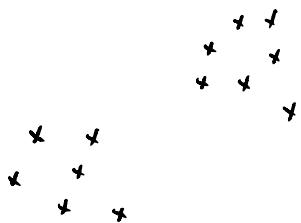
CS-GY 6033
Design and Analysis of Algorithms I

Lecture 8 | Part 13

MSTs and Clustering

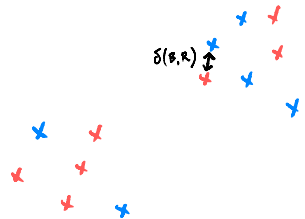
Clustering

Goal: identify the groups in data. Example:



Clustering, Formalized

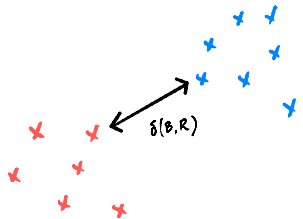
- We frame as an optimization problem.
  - GIVEN: n data points.
  - GOAL: assign color to each point (red or blue) to maximize the distance between the closest pair of red and blue points.



Bad Clustering

Clustering, Formalized

- We frame as an optimization problem.
  - GIVEN: n data points.
  - GOAL: assign color to each point (red or blue) to maximize the distance between the closest pair of red and blue points.



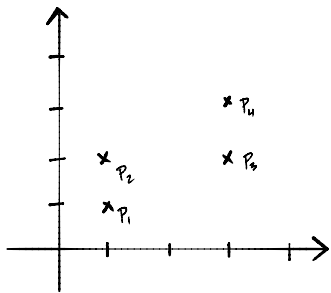
Good Clustering

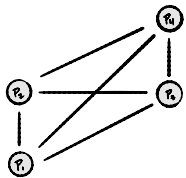
Brute Force Solution

- Try all possible assignments; return best.
- If there are n data points, there are $\Theta(2^n)$ assignments.
- Exponential time; very slow. Practical only for ~ 50 data points.
- Instead, we will turn it into a graph problem.

Distance Graphs

- Given n data points, $p_1, p_2, \ldots, p_n$, create complete graph with $V = \{p_1, \ldots, p_n\}$.
- Set weight of edge (p_i, p_j) = dist(p_i, p_j).
- The result is a weighted, undirected **distance graph**.



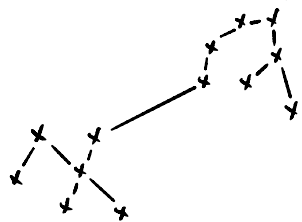


Main Idea

We can always think of a set of points in a (metric) space as a weighted distance graph. This is a **very** important idea, because it allows us to use our graph algorithms!

Clustering with MSTs

- Given n data points and a number of clusters, k:
  - Create distance graph G.
  - Run Kruskal's Algorithm on G until there are only k components.



  - The resulting connected components are the **clusters**.
  - This is known as **single-linkage clustering**.

Example

Single-Linkage Clustering

- Time complexity of single-linkage is determined by Kruskal's Algorithm: $\Theta(E \log E)$.
- Since distance graph is complete, $E = \Theta(V^2)$, and so

$$\Theta(E \log E) = \Theta(V^2 \log V) = \Theta(n^2 \log n)$$

- Practically, can cluster ~ 10,000 points.

Summary

- We started the quarter with a brute force solution.
  - Took $\Theta(2^n)$ time, only feasible for a few dozen points.
- We've now reframed the problem using graph theory.
  - Now only $\Theta(n^2 \log n)$ time!
  - Feasible for tens of thousands of points.